M1: Approach · AI Incident Report Assistant

Megathon X · Qualcomm Problem Statement 1 · Team: Shubham Gupta, Sarthak Chaudhari

1. Problem statement and our understanding

Reporting an incident today means opening a portal and typing every field. The problem asks for an assistant where the reporter **just speaks** ("There was a water leak near Conference Room 2 around 10 AM. Maintenance has already isolated the area.") and the app fills a structured report: **speak → understand → extract → review → submit**, with the result shown on screen and spoken back. **Every AI model must run on the Snapdragon device (Qualcomm QIDK), with no cloud APIs.**

What we take from it:

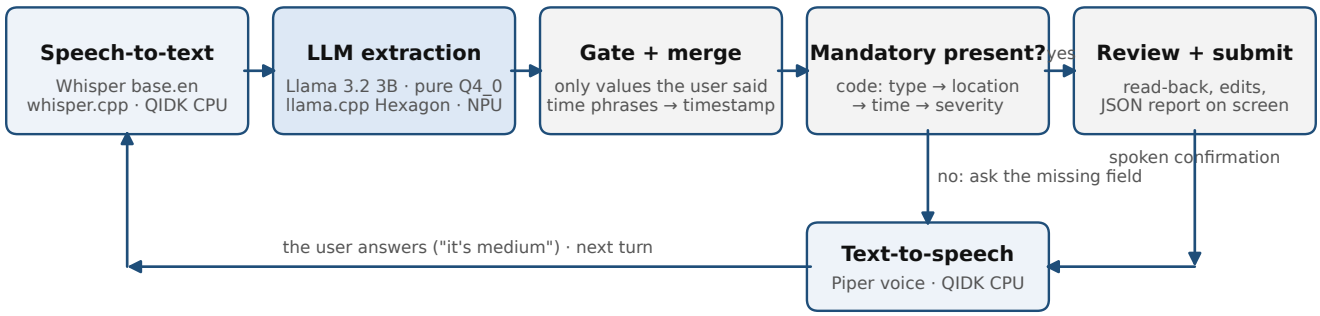
- **It is multi-turn slot filling, not one-shot extraction.** People leave things out, answer follow-up questions with one word ("medium"), and correct themselves ("actually it was Lab 4"). The form has to stay right across the whole conversation.
- **"Mandatory fields present?" is a decision, so code should make it, not the model.** The same goes for which question to ask next, whether a value was really said, and when submitting is allowed.
- **On the device, output length is the cost.** On the QIDK's Hexagon NPU, generating tokens took 82–89% of model time in our earlier project (FieldMind, measured on this board). So the model writes a short tool call, and every sentence the user hears is written by code.
- **Judged on:** on-device Snapdragon acceleration · natural speech, not fixed commands · speech recognition accuracy · extraction accuracy · mapping to the right fields.

The report form: the problem statement's 10 fields. Mandatory: incident type, severity, location, date & time, description. Optional: persons involved, equipment / asset, immediate actions taken, witnesses, recommended follow-up. 15 incident types (fire, electrical, water leak, gas leak, chemical, slip/trip, fall from height, struck by, caught in, equipment failure, lift, structural, medical, security, other).

2. Idea

A **single-agent, fully on-device voice assistant**. Whisper turns speech into text; **one grammar-constrained LLM call per turn, on the NPU,** turns the utterance into a tool call (update_report / submit_report / cancel_report / no_op); **plain code** checks it against what the user actually said, merges it, decides what is still missing and asks for it; a Piper voice speaks the reply. In one line: **the model extracts, code decides.** It builds on FieldMind, our earlier project that ran LLMs on this same QIDK's Hexagon NPU with llama.cpp (every layer on the NPU, verified from the server's own log).

3. Architecture and pipeline



Each model has a fixed processor, so no two stages compete for the NPU:

stage	model	runtime	runs on
Speech-to-text	Whisper base.en (tiny / small compared)	whisper.cpp server	QIDK CPU
Extraction	Llama 3.2 3B Instruct, pure Q4_0 (Qwen3 1.7B as faster backup)	llama.cpp, Hexagon backend, llama-server	QIDK NPU (HTP0)
Text-to-speech	Piper en_US lessac medium	sherpa-onnx	QIDK CPU
Dialog logic, UI	none (plain Python, standard library only)	HTTP server + web page	laptop now; board screen (stretch)

- **One LLM call per turn, always valid.** The prompt holds fixed instructions (~600 tokens, reusable across turns), the form so far, the field just asked about, and the utterance. The reply is a tool call with one-letter keys, constrained by a **JSON schema that llama-server compiles into a grammar**: enums can only hold allowed values and text fields have a maximum length. (FieldMind: free JSON from 1–3B models broke in 25–100% of answers; with a grammar, 0.)
- **Code after the model.** Drop any value whose words are not in the user's speech (numbers must match exactly: "Room 3" never passes for "Room 2"); corrections overwrite; time phrases ("around 10", "quarter past two", "8 last night") become timestamps; ask the first missing mandatory field with a fixed question; read back and wait for "submit"; refuse to submit while a mandatory field is empty. The description is the user's own words, so it cannot be invented. Follow-up actions come from a fixed list per incident type. A severity below the usual minimum for the type is questioned at review, never changed silently.
- **NPU, proven.** Models are built as pure Q4_0 (published "Q4_0" files carry other tensor types); the board refuses any model file whose hash doesn't match. Every launch is checked from the server's own log: all layers offloaded to HTP0 and a nonzero HTP0 buffer (Llama 3B: 29/29 layers). Measured on this board in FieldMind: **986 tok/s prefill, 21 tok/s decode** for Llama 3.2 3B (700-token prompt). Target: under 6 s per spoken turn end to end (to be measured).
- **Deployment.** Tier 1: all three models on the QIDK, UI on a laptop over USB (adb). Tier 2 (stretch): the UI on the board's own screen, plus an airplane-mode demo.

4. Planned testing methodology

what we test	how	judging criterion
Extraction and field mapping	**Text eval set: 65 scripted dialogs, 250 user turns**, a correct tool call for every turn. 52 dialogs are rewrites of real public incident reports (OSHA accident summaries, Indian news), 13 hand-written. Traps: negations, corrections, one-word answers, premature submit, noise. **Teacher-forced** (each turn from the correct form so far): tool accuracy, per-field accuracy, values the model added that the user never said.	extraction accuracy; correct mapping
The conversation	**Free-running:** each dialog through the real dialog loop from an empty form; the final form against the correct one; did it end right (submitted / cancelled).	natural conversational speech
Speech recognition	**36 dev turns × 3 speakers**, plus 10 in a noisy room, chosen where STT is likeliest to hurt (numbers, times, names, accents, "not"/"actually"). Word error rate and errors on key words; then **the same turns typed vs spoken** through extraction, to show what STT costs. Whisper tiny / base / small, with and without a vocabulary hint.	speech recognition accuracy
On-device and latency	Every run gated by the NPU log check; per-stage time from the servers themselves (STT, LLM prefill, LLM decode, code, TTS); chip temperature before and after.	on-device Snapdragon acceleration

- **Rules for the numbers.** Every figure is labelled BOARD or LAPTOP, estimates are marked as estimates, and nothing is invented. Each eval runs twice (replies at temperature 0 still vary between server launches). Tuning uses the dev split only; the 21-dialog test split stays locked until the decision rules (3B vs 1.7B, Whisper size, vocabulary hint) are written down.
- **Status at M1.** The dialog core, UI, voice input/output, evaluation harness and board bring-up scripts are built, with 138 automated tests passing (scorer self-check: 100% on the correct answers). Next: the full run on the QIDK, then the measured results for the final submission.